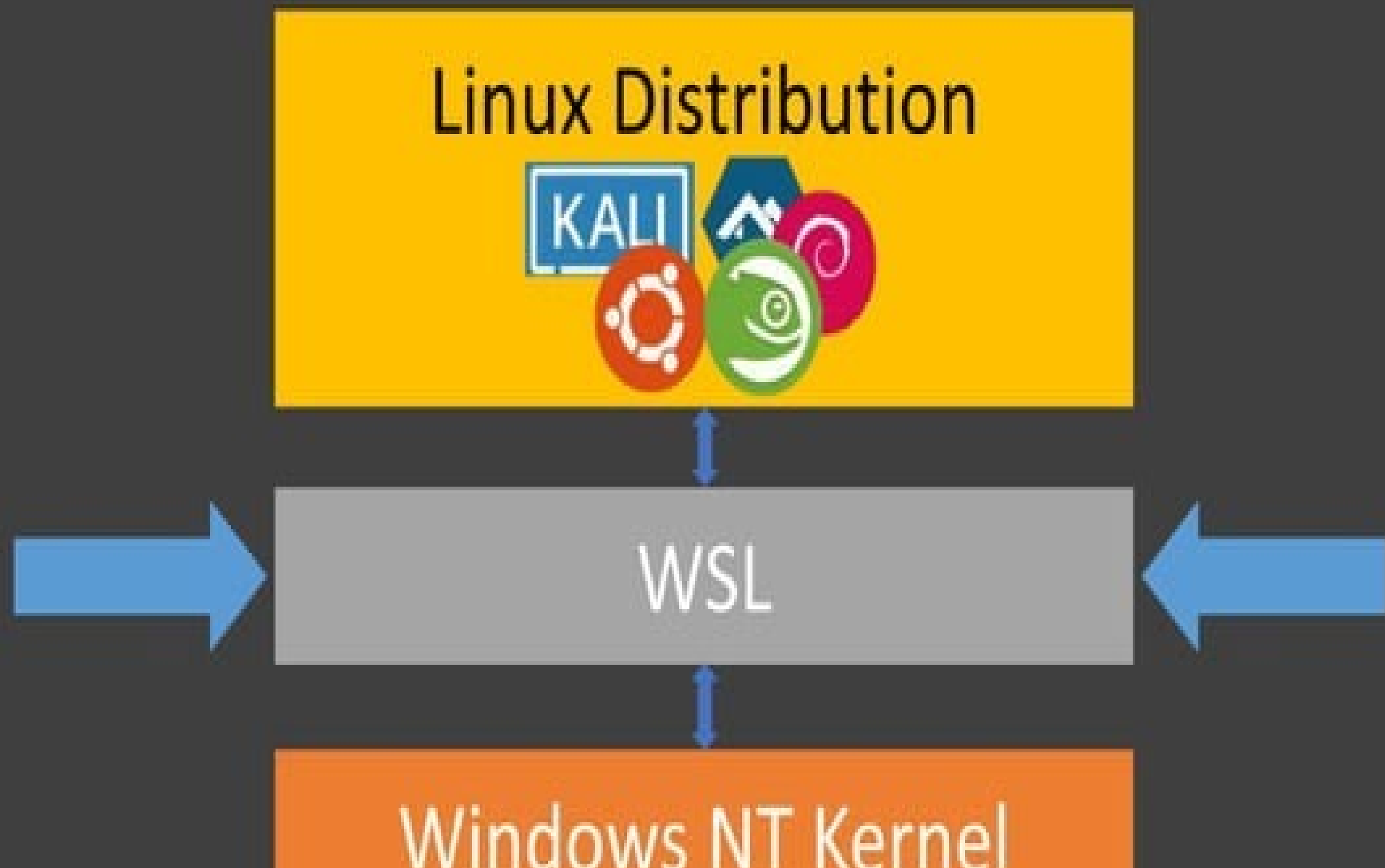

Các công nghệ lập trình hiện đại

Docker và WSL2

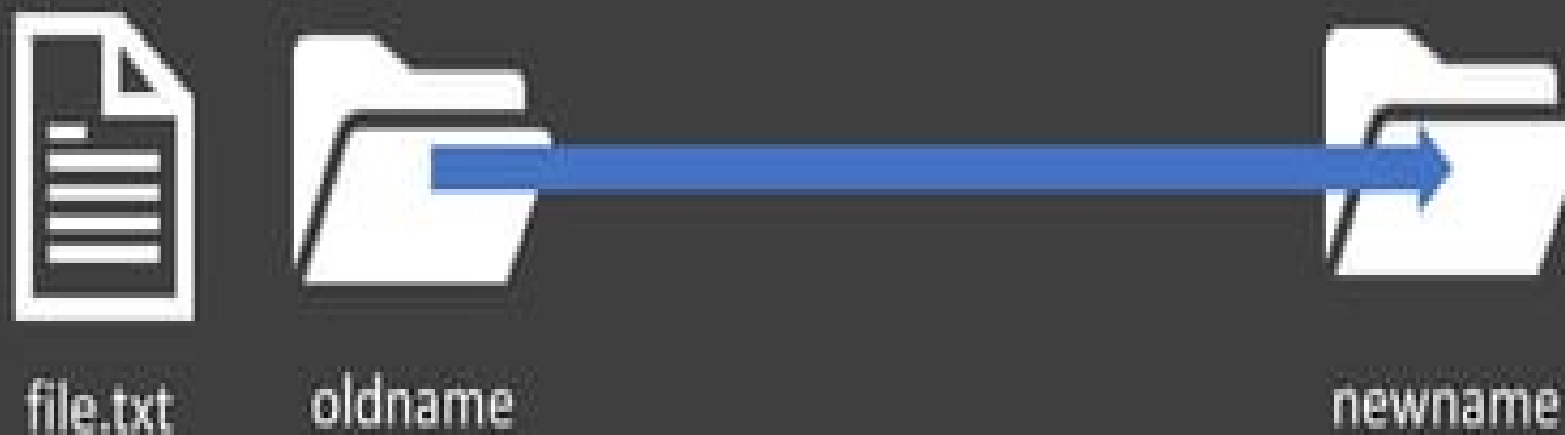
TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

WSL 2: Windows Subsystem for Linux v2

How does this work? – WSL 1 Architecture



Renaming a directory with an open file



Windows

```
OpenFile("C:\\oldname\\file.txt", ...);
```

```
MoveFile("C:\\oldname",  
"C:\\newname");
```

ERROR_ACCESS_DENIED

Linux

```
open("/oldname/file.txt", ...);
```

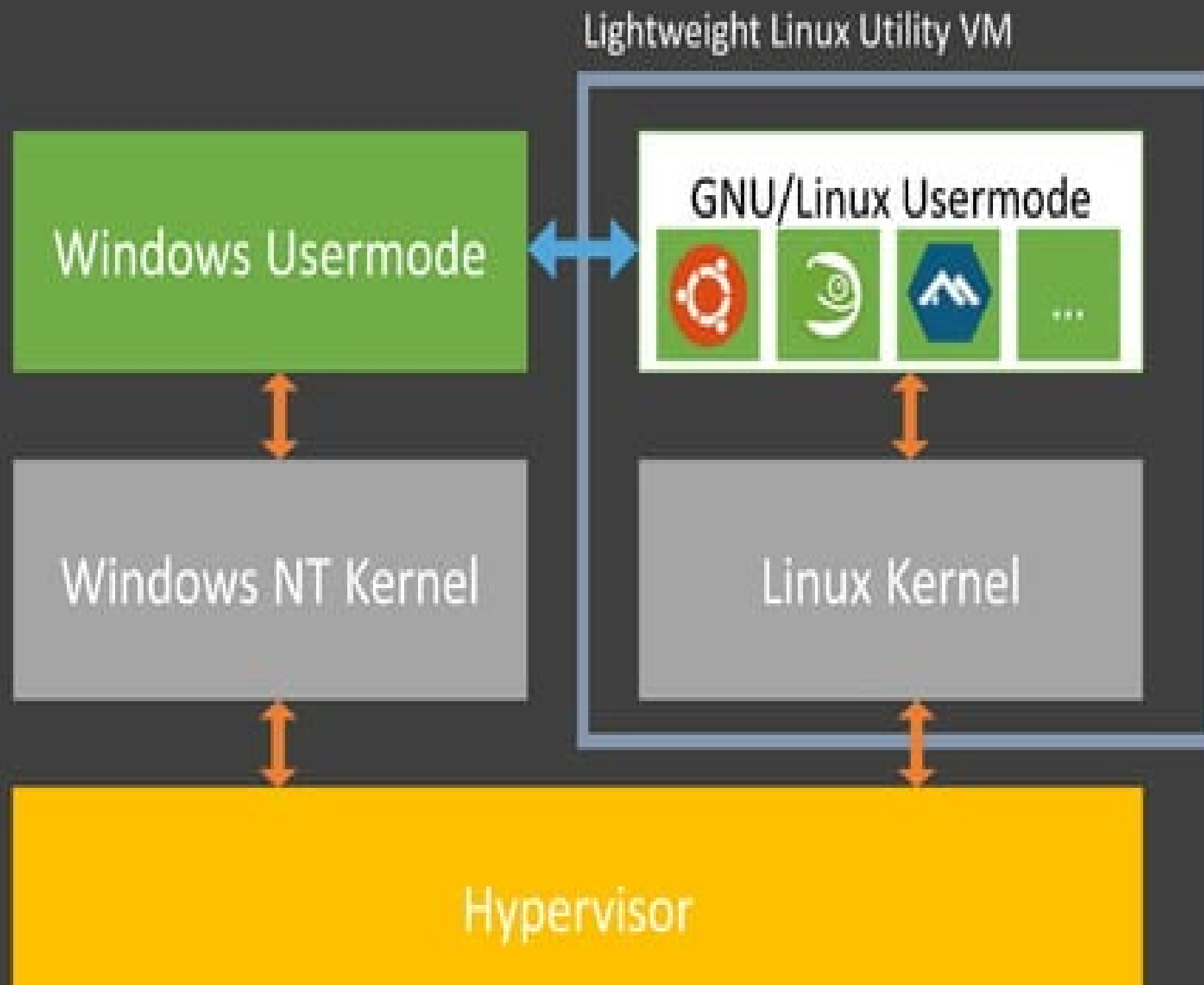
```
rename("/oldname", "/newname");
```

SUCCESS

<https://www.flickr.com/photos/jev55/8093515962/>



WSL 2 Architecture Overview



Traditional VM

vs

WSL 2

- Isolated
- Slower to boot
- Large memory footprint
- You need to manage it

- Integrated
- Fast to boot (~1 second)
- Small memory footprint
- Only runs when you need it

File System Performance

WSL 2 speed comparison test WSL 1 on a Surface Laptop

git clone

2.5x

faster

npm install

4.7x

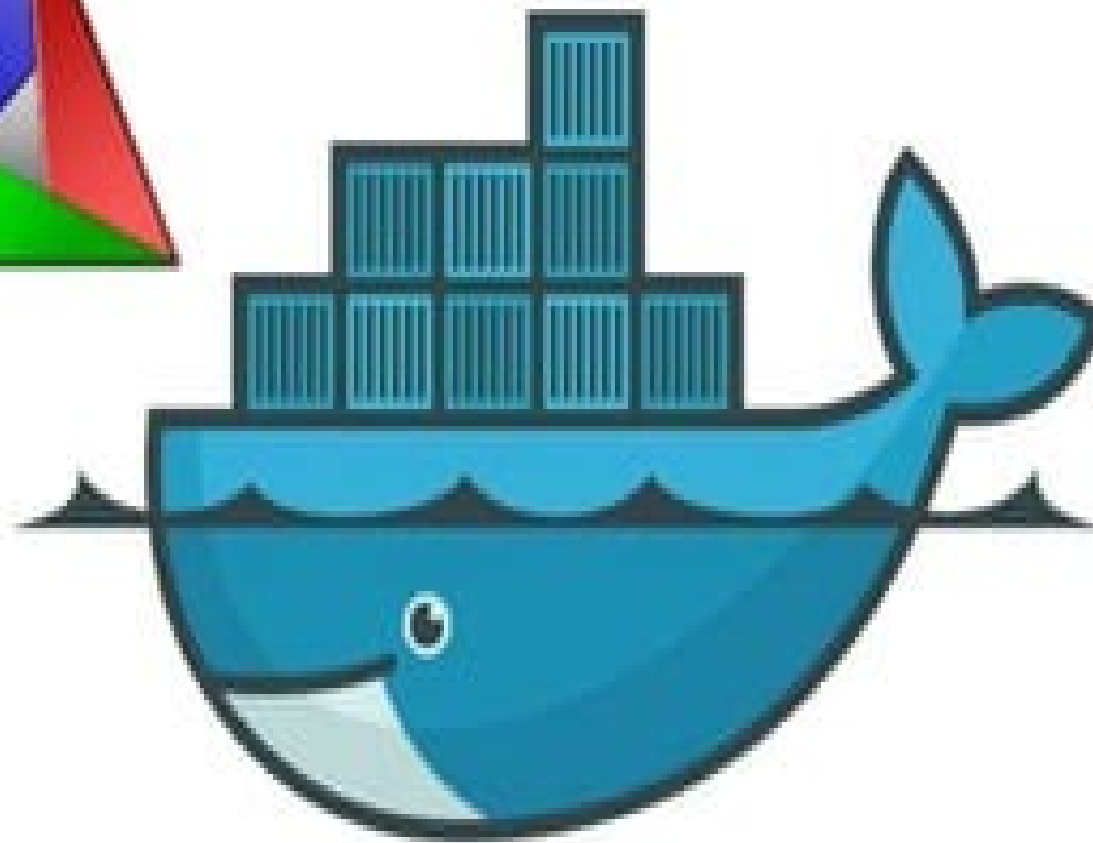
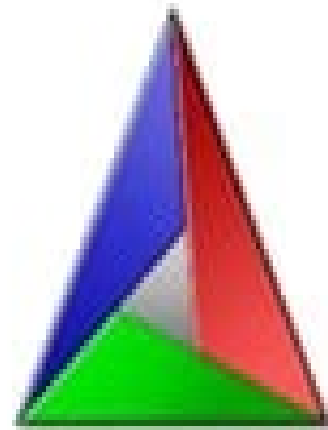
faster

cmake

3.1x

faster

Full System Call Compatibility



docker





 [Desktop, Windows](#)

Docker ❤️ WSL 2 – The Future of Docker Desktop for Windows



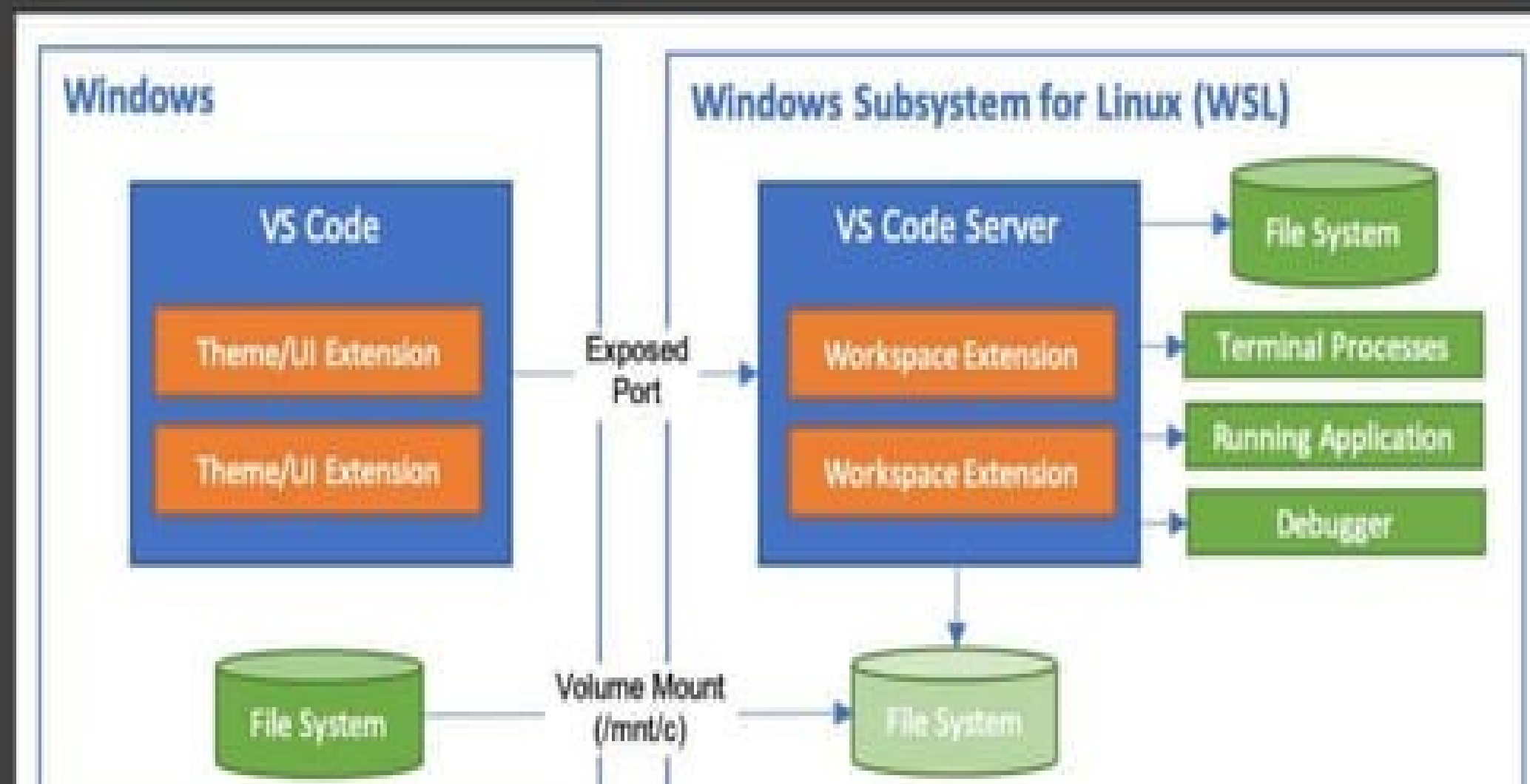
SIMON FERGUEL

Jun 16

One of Docker's goals has always been to provide the best experience working with containers from a Desktop environment, with an experience as close to native as possible whether you are working on Windows, Mac or Linux. We spend a lot of time working with the software stacks provided by Microsoft and Apple to achieve this. As part of this work, we have been closely monitoring Windows Subsystem for Linux (WSL) since it was introduced in 2016, to see how we could leverage it for our products.

Visual Studio Code + WSL

- Visual Studio Code Remote
 - Build, run, and debug your Linux applications directly from VS Code



```
C:\sharpcode> echo "Hello" | clip.exe
```

```
C:\sharpcode> Get-Clipboard
```

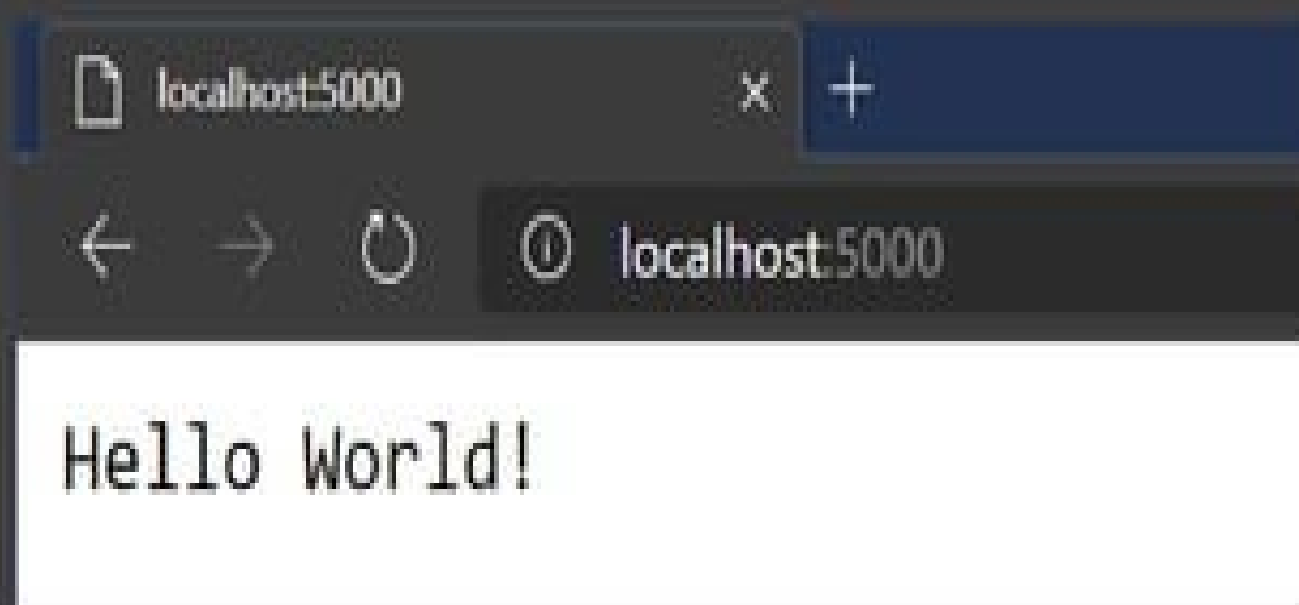
```
Hello
```

```
$ echo "Hello from $(uname)" | clip.exe
```

```
C:\sharpcode> Get-Clipboard
```

```
Hello from Linux
```

```
$ dotnet new mvc --name wsl-web
$ cd dotnet-web/
$ dotnet run
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: https://localhost:5001
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://localhost:5000
```



Windows Features

Turn Windows features on or off

To turn a feature on, select its check box. To turn a feature off, clear its check box. A filled box means that only part of the feature is turned on.

- ☒ SMB Direct
- ☐ Telnet Client
- ☐ TFTP Client
- ☒ Virtual Machine Platform
- ☐ Windows Hypervisor Platform
- ☐ Windows Identity Foundation 3.5
- ☒ Windows PowerShell 2.0
- ☐ Windows Process Activation Service
- ☐ Windows Projected File System
- ☐ Windows Sandbox
- ☒ Windows Subsystem for Linux
- ☐ Windows Defender
- ☒ Work Folders Client

Provides services and environments for running native user-mode Lin



Run Linux on Windows

Install and run Linux distributions side-by-side on the Windows Subsystem for Linux (WSL).



ubuntu®



fedora
remix



Alpine WSL

Docker under WSL 2

Agenda

- Containers are NOT VMs
- Working with Docker (Build, Ship, Run)
- But Why?
- Getting started
- Q & A

Containers are not VMs



Docker containers are NOT VMs

- Easy connection to make
- Fundamentally different architectures
- Fundamentally different benefits

VMs

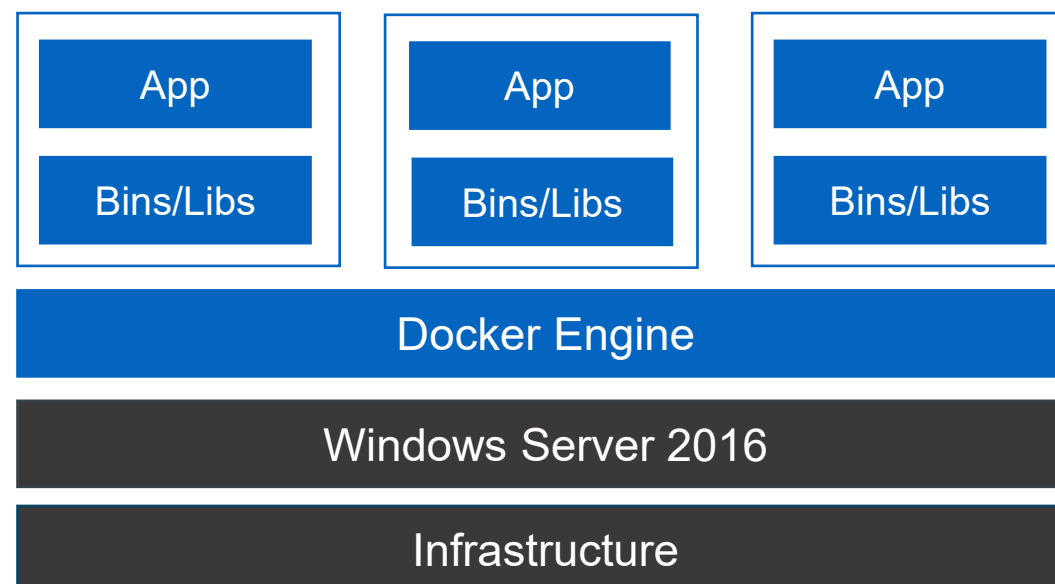


Containers

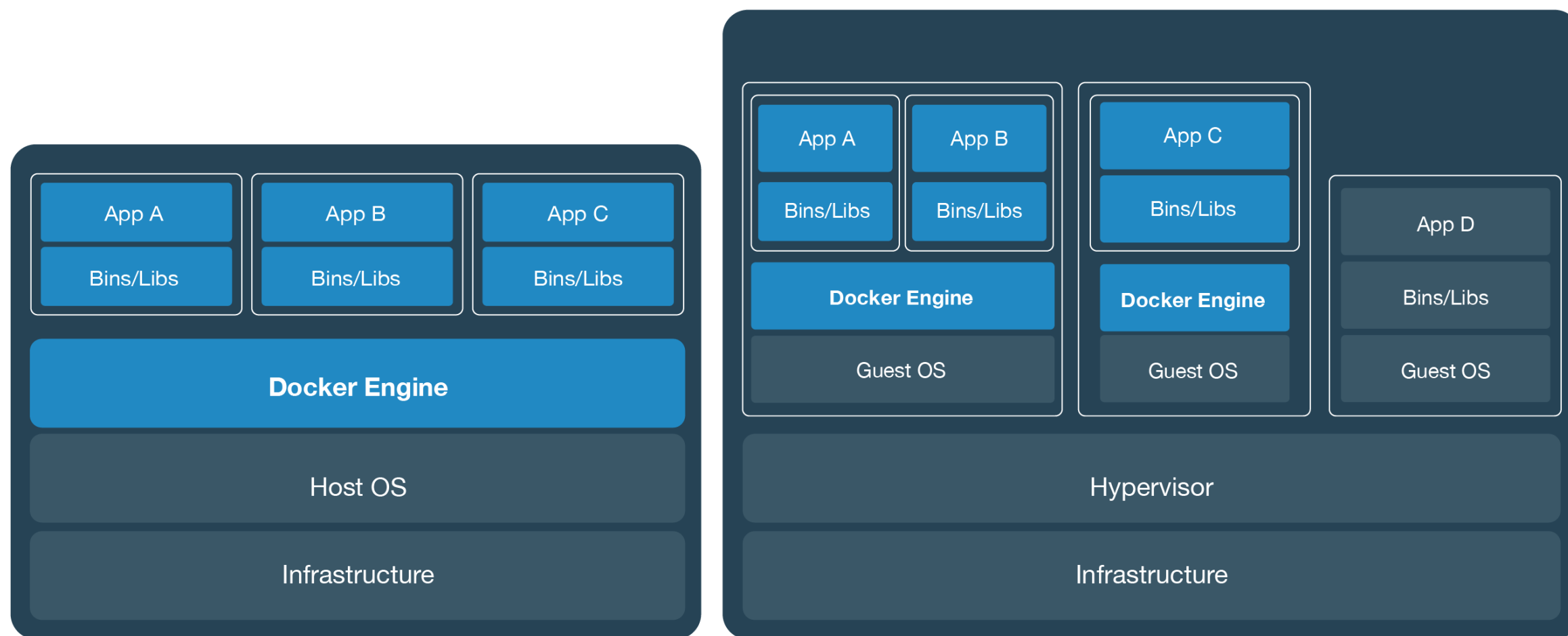


Docker + Windows Server = Windows Containers

- Native Windows containers powered by Docker Engine
- Windows kernel engineered with new primitives to support containers
- Deep integration with 2+ years of engineering collaboration in Docker Engine and Windows Server
- Microsoft is top 5 Docker open source project contributor and a Docker maintainer

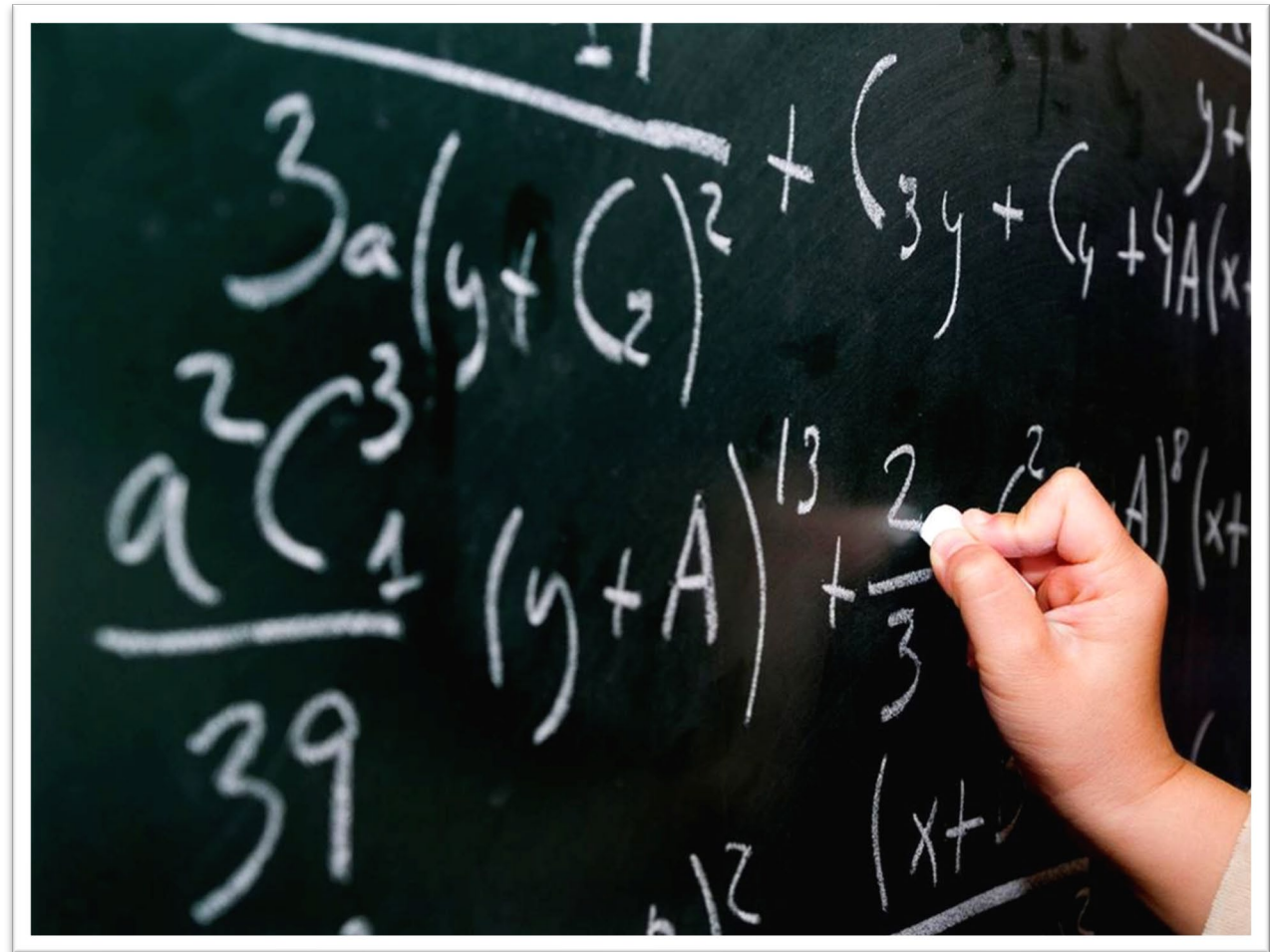


They're different, not mutually exclusive



Variables to Consider

- Performance
- Security
- Scalability
- Existing Skillsets
- Costs
- Etc.



http://people-equation.com/do-your-words-encourage-or-deflate/math-equation_chalkboard/

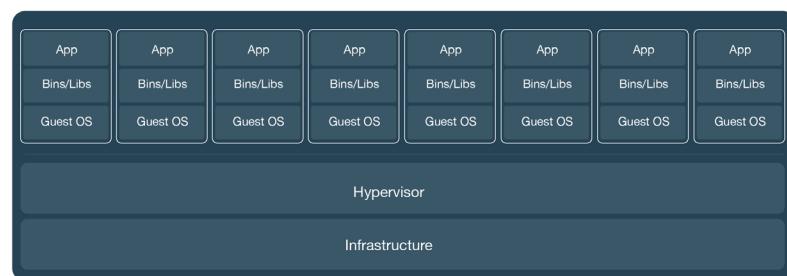
Container Consolidation Testing

- **How can containers help organizations optimize hardware utilization?**
- Testing done by HPE, Docker and Industry Consultant
- Components:
 - Docker CS Engine 1.12.3
 - VMware ESXi 6
 - SysBench 1.0 (Nov 2016)
 - RHEL 7.2
 - HPE ProLiant DL360 Gen 9 servers with HPE 3PAR StorServ 8200 SSD Storage

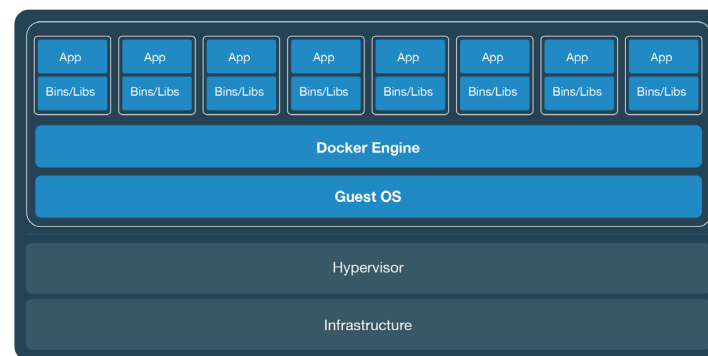
Testing Scenarios

- Measure SysBench performance across 3 configurations

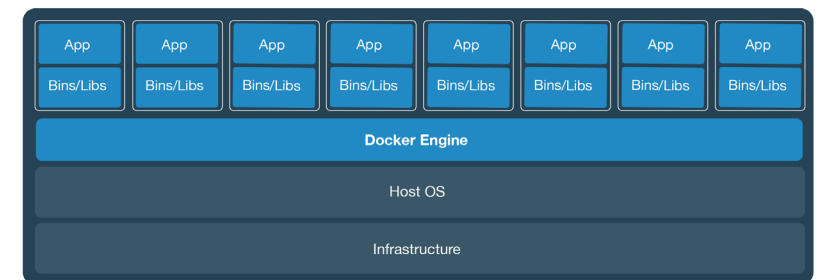
Need to replace this graphic w/ the one w/ 8 boxes



Virtual Machines



Virtual Machines with Containers



Containers on Bare Metal

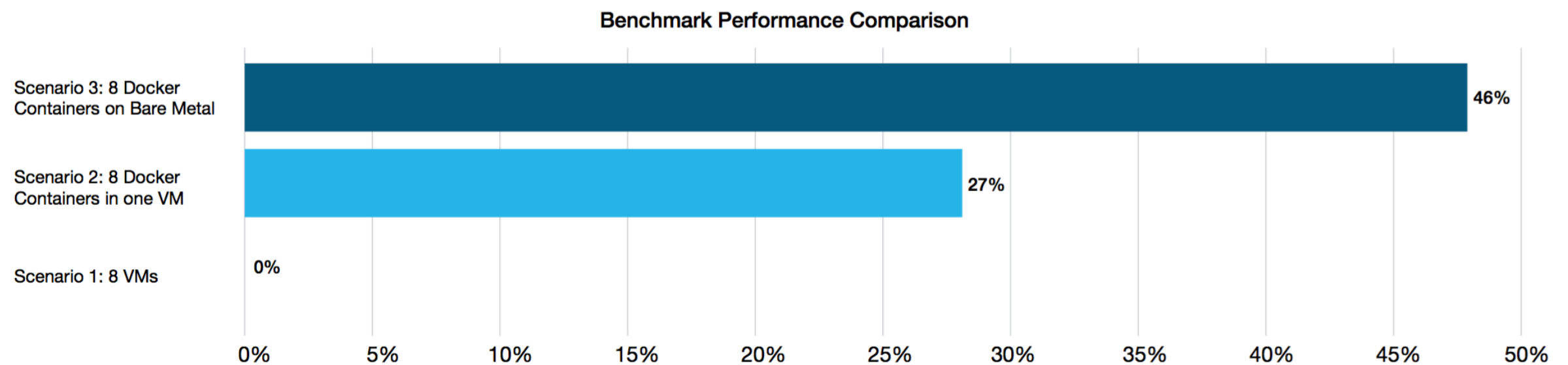
Scenario 1:
8 VMs

Scenario 2:
1 VM w/ 8 Containers

Scenario 3:
8 Containers on Bare Metal

Results

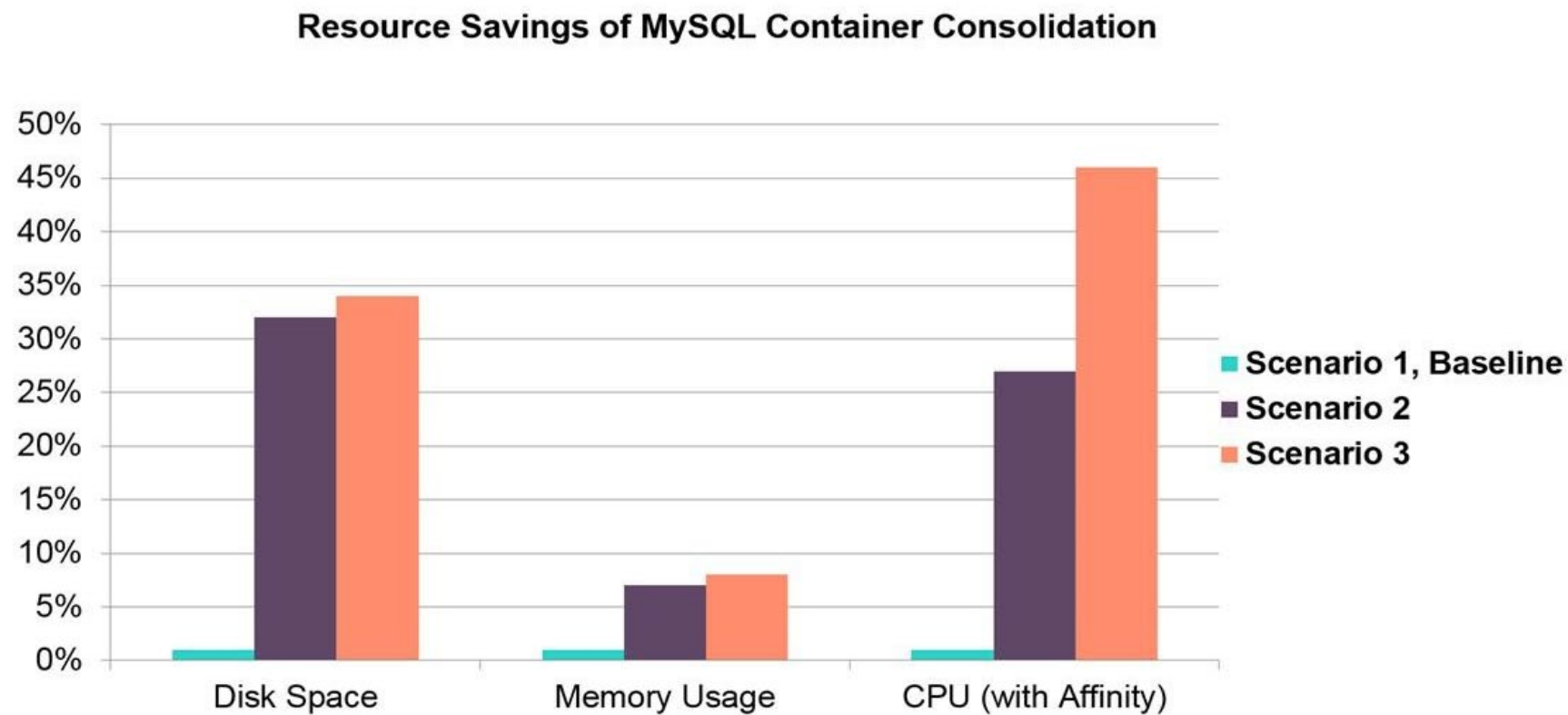
- Moving from VMs to containers increases performance 27% to 46%



Results are after VM and Container Tuning

Additional Savings

- Docker allowed for savings in memory and disk as well



Key Learnings

- Docker containers increase performance and flexibility

1

Plan for Higher Density

2

Bare Metal or Bigger VMs

3

Tune To Optimize

Build, Ship, Run, Any App Anywhere

From Dev



Any App



Any OS



Windows



Linux

Anywhere



Physical



Virtual



Cloud

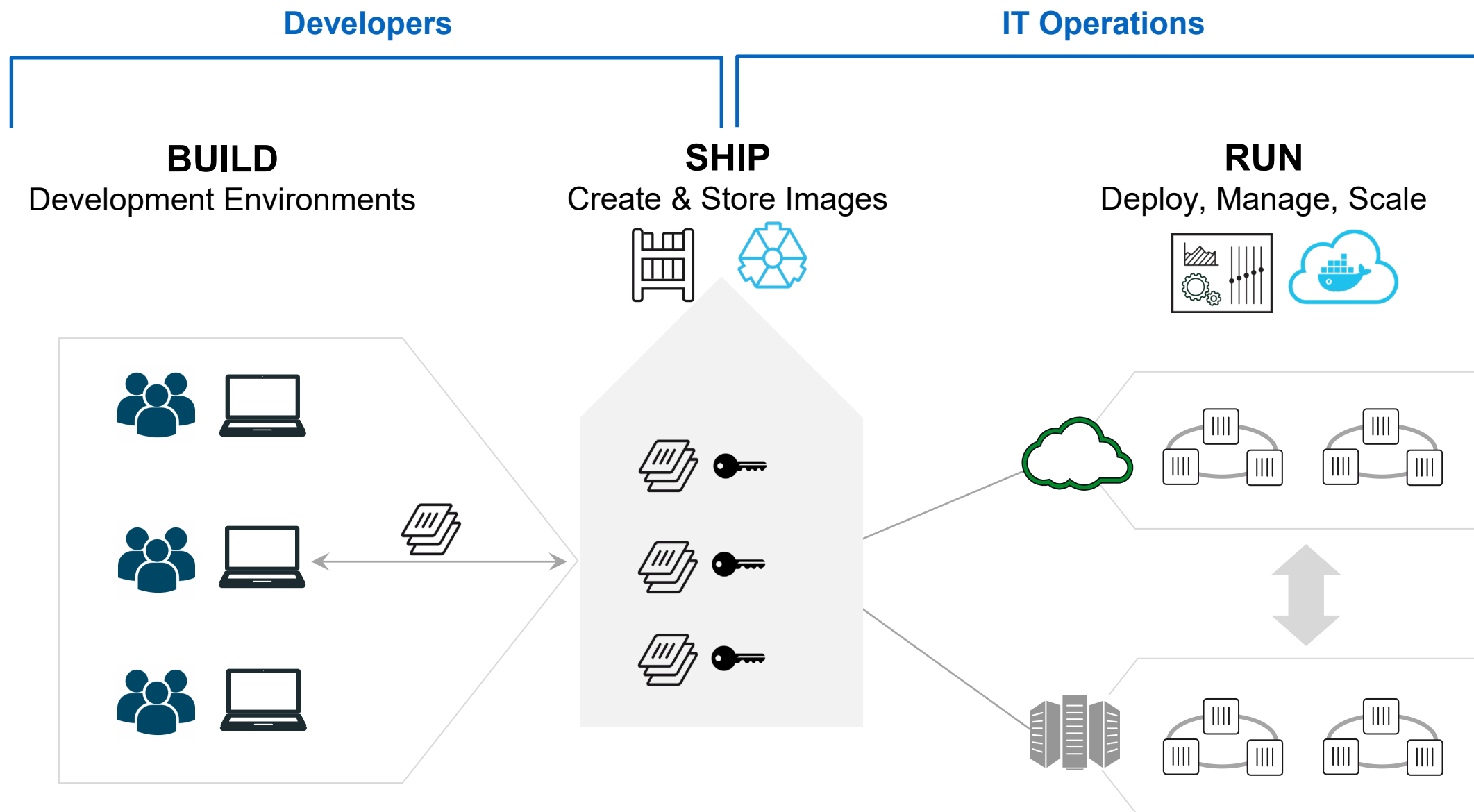
To Ops



Build, Ship, and Run

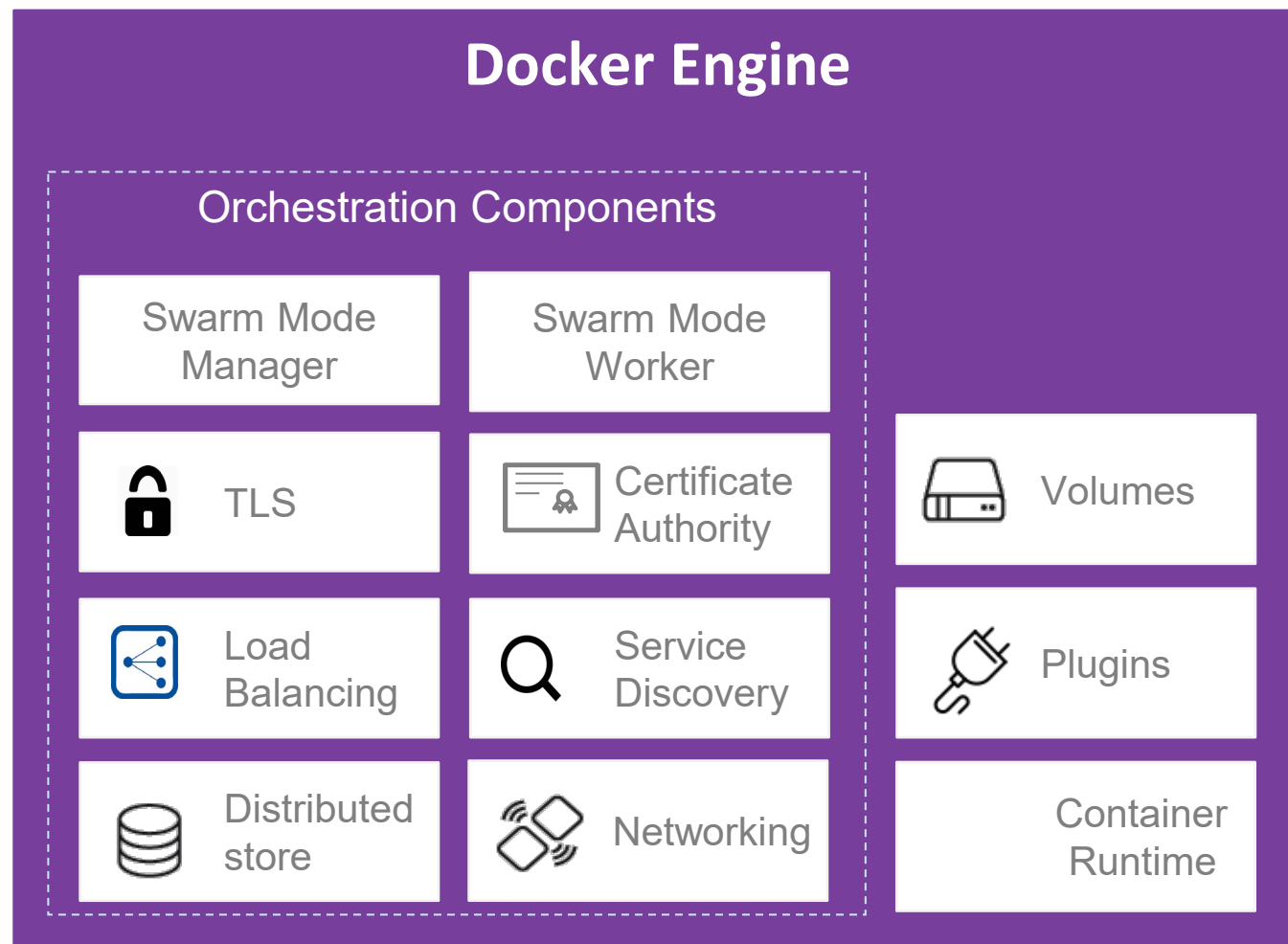


Put it all together: Build, Ship, Run Workflow



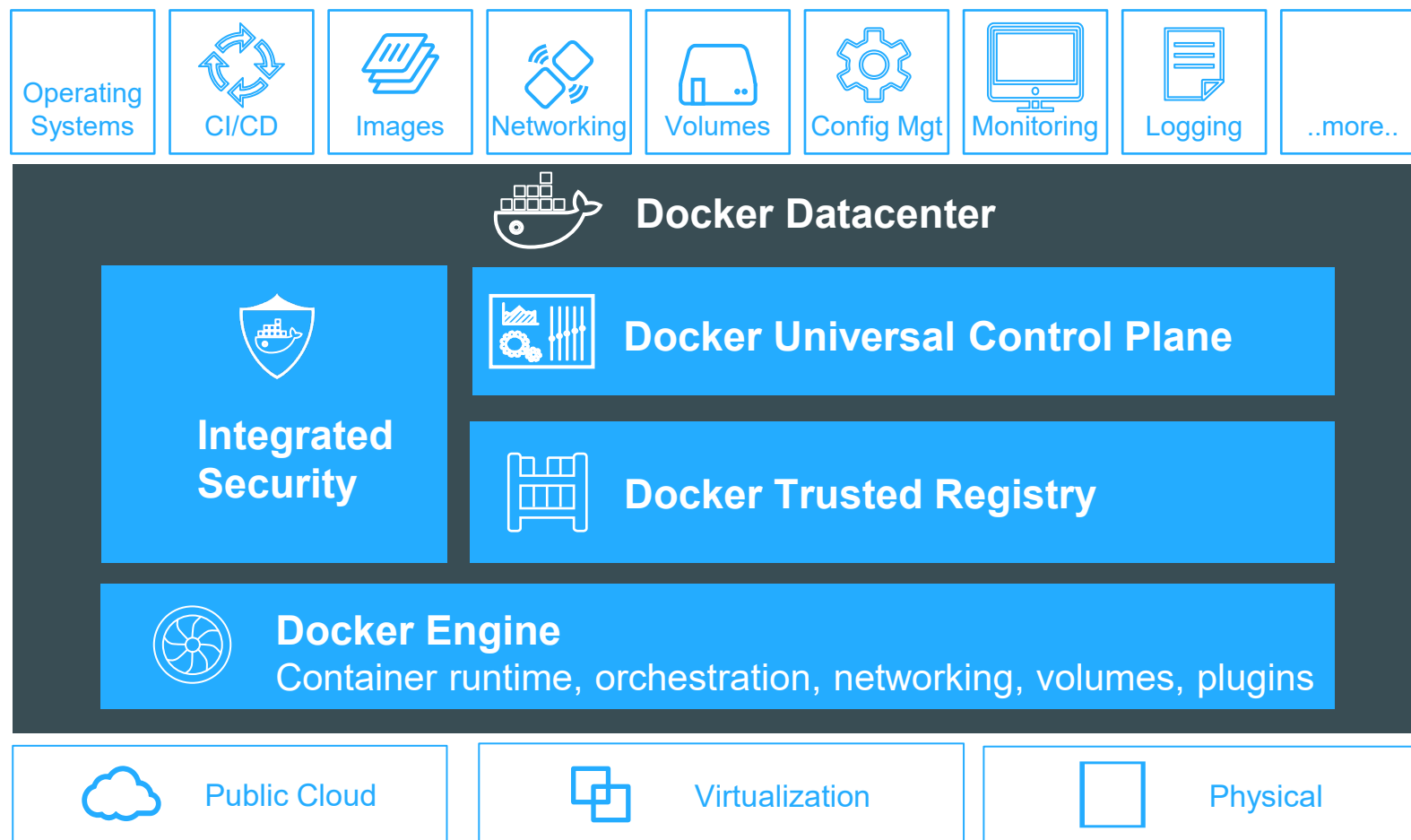
The building block: Docker Engine 1.12

Built in orchestration with scheduling, networking and scheduling

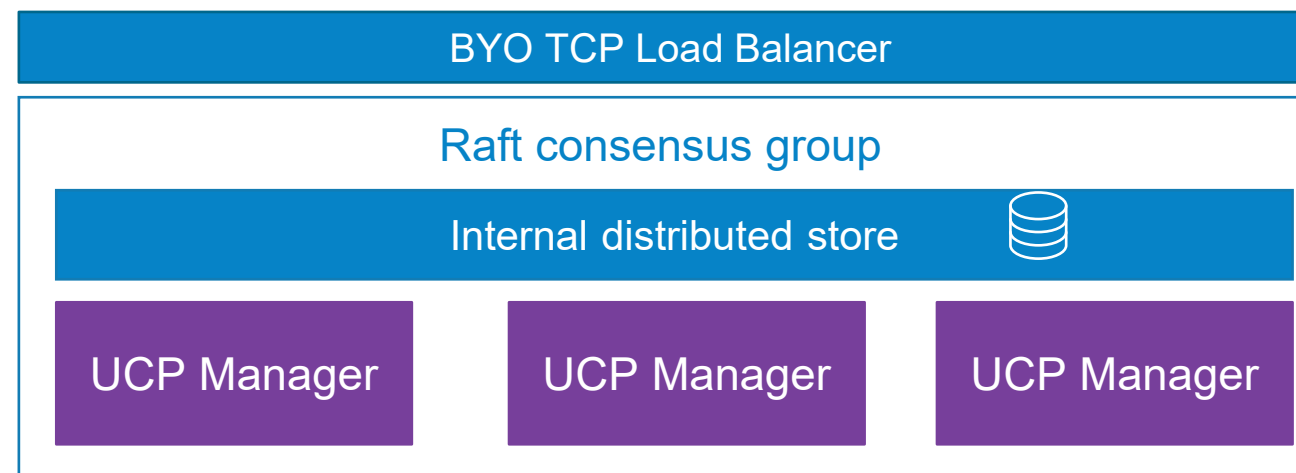


- Powerful yet simple, built in orchestration
- Declarative app services
- Built in container centric networking
- Built in default security
- Extensible with plugins, drivers and open APIs

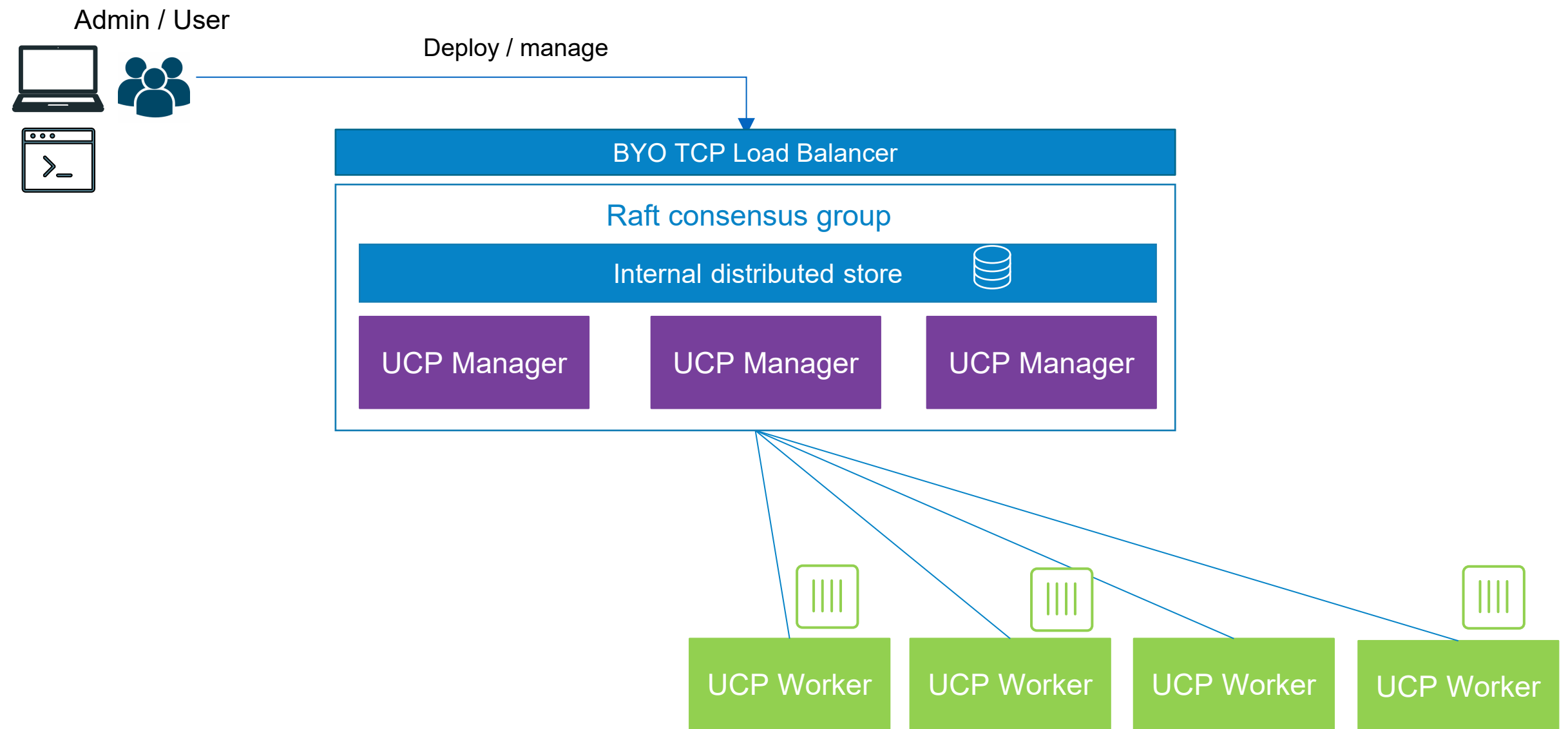
Docker Datacenter platform



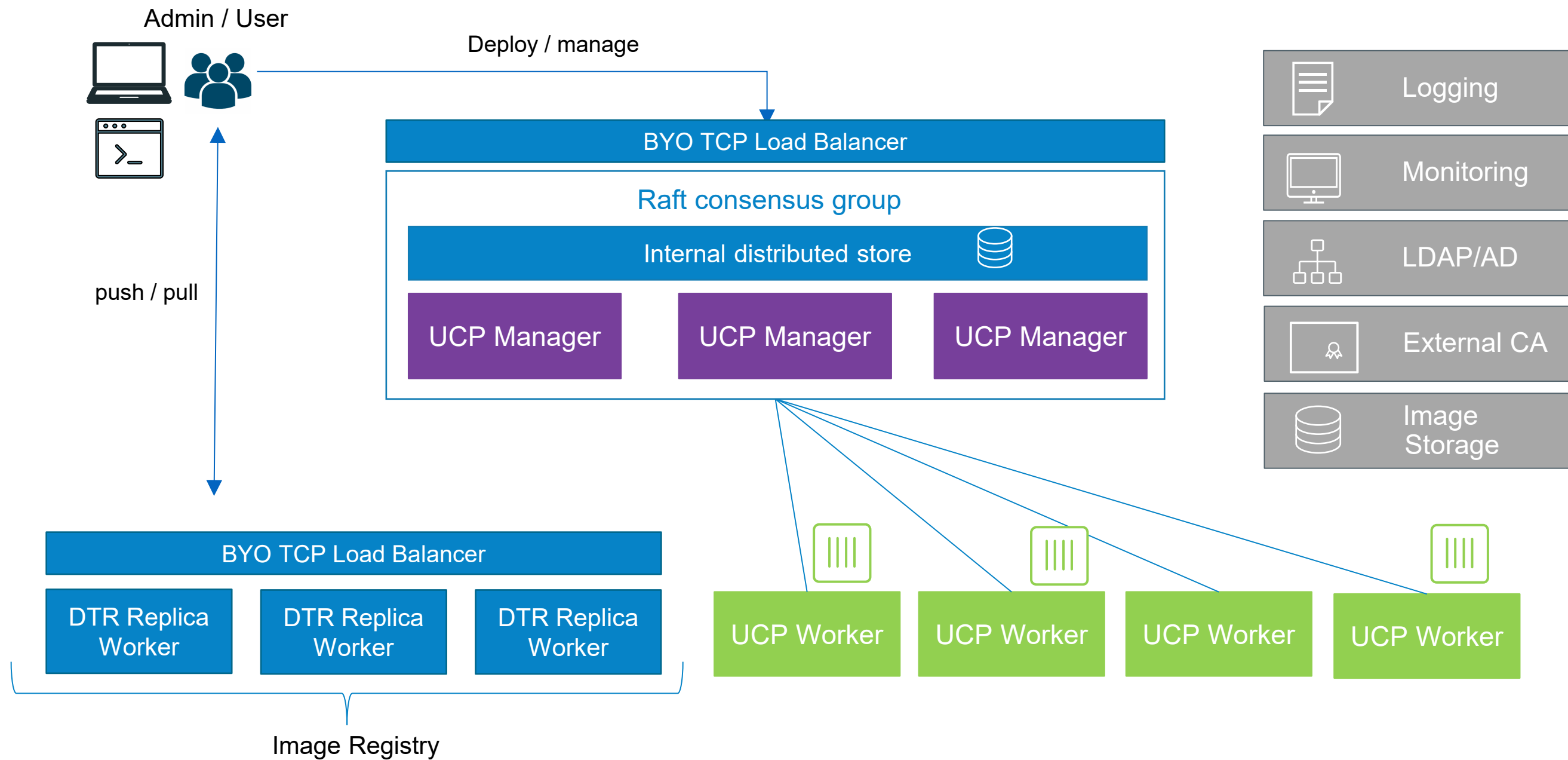
Docker Datacenter Architecture



Docker Datacenter Architecture



Docker Datacenter Architecture

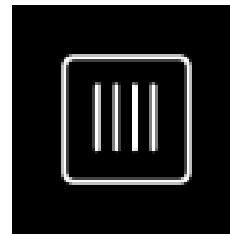


Some Docker vocabulary



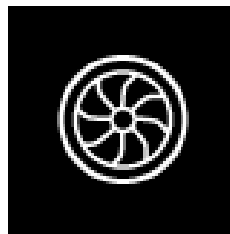
Docker Image

The basis of a Docker container. Represents a full application



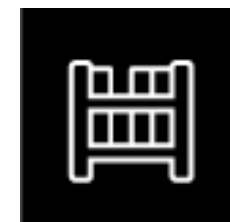
Docker Container

The standard unit in which the application service resides and executes



Docker Engine

Creates, ships and runs Docker containers deployable on a physical or virtual, host locally, in a datacenter or cloud service provider



Registry Service (Docker Hub or Docker Trusted Registry)

Cloud or server based storage and distribution service for your images

Basic Docker Commands

```
$ docker pull mikegcoleman/catweb:1.0
```

```
$ docker images
```

```
$ docker run -d -p 5000:5000 --name catweb mikegcoleman/catweb:latest
```

```
$ docker ps
```

```
$ docker stop catweb (or <container id>)
```

```
$ docker rm catweb (or <container id>)
```

```
$ docker rmi mikegcoleman/catweb:latest (or <image id>)
```

```
$ docker build -t mikegcoleman/catweb:2.0 .
```

```
$ docker push mikegcoleman/catweb:2.0
```


Dockerfile – Linux Example

```
1 # our base image
2 FROM alpine:latest
3
4 # Install python and pip
5 RUN apk add --update py-pip
6
7 # upgrade pip
8 RUN pip install --upgrade pip
9
10 # install Python modules needed by the Python app
11 COPY requirements.txt /usr/src/app/
12 RUN pip install --no-cache-dir -r /usr/src/app/requirements.txt
13
14 # copy files required for the app to run
15 COPY app.py /usr/src/app/
16 COPY templates/index.html /usr/src/app/templates/
17
18 # tell the port number the container should expose
19 EXPOSE 5000
20
21 # run the application
22 CMD ["python", "/usr/src/app/app.py"]
```

- Instructions on how to build a Docker image
- Looks very similar to “native” commands
- Important to optimize your Dockerfile

Dockerfile – Windows Example

19 lines (15 sloc) | 832 Bytes

Raw

Blame

History



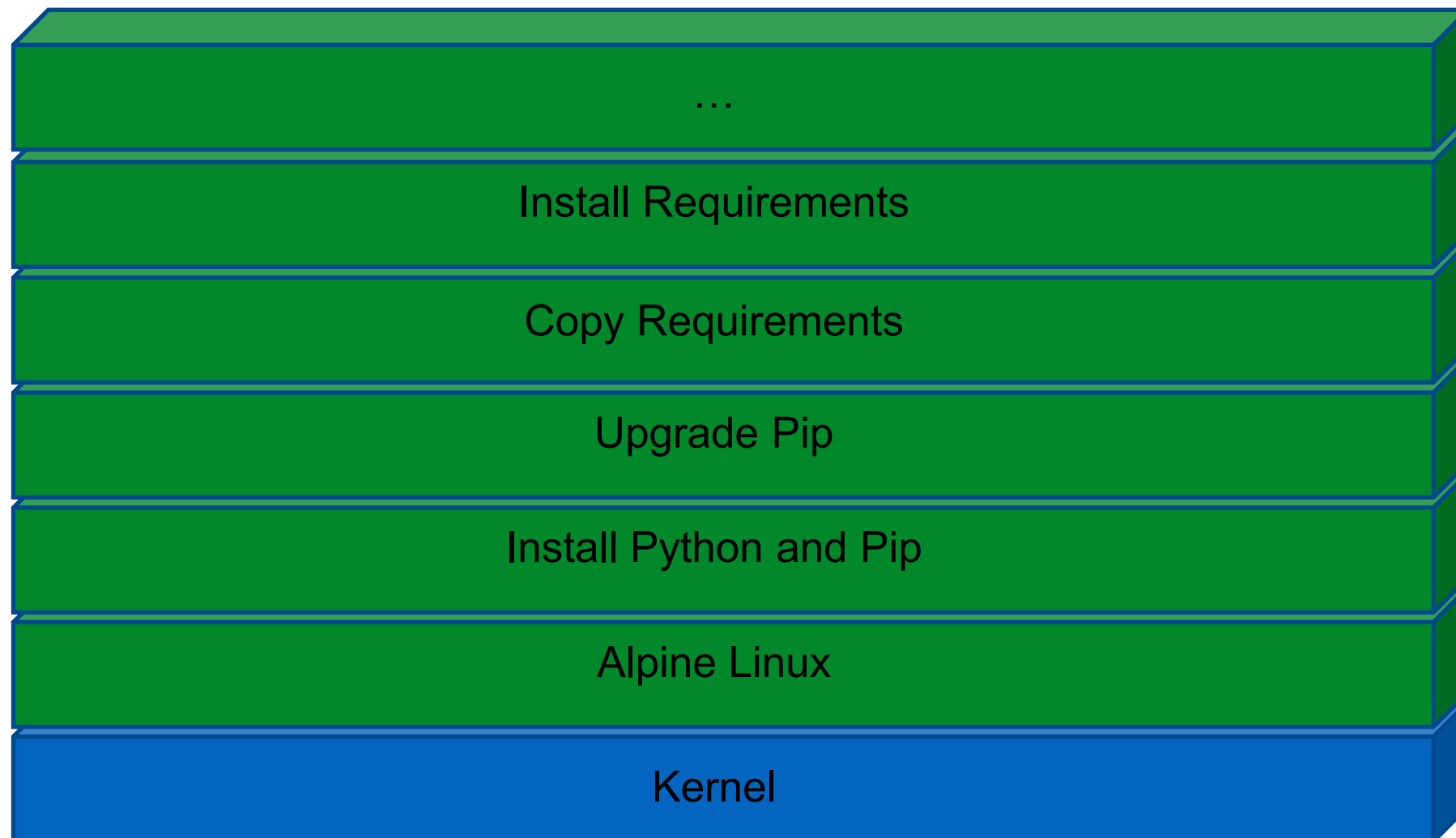
```
1 FROM microsoft/windowsservercore
2
3 ENV NPM_CONFIG_LOGLEVEL info
4 ENV NODE_VERSION 6.5.0
5 ENV NODE_SHA256 0c0962800916c7104ce6643302b2592172183d76e34997823be3978b5ee34cf2
6
7 RUN powershell -Command \
8     $ErrorActionPreference = 'Stop' ; \
9     (New-Object System.Net.WebClient).DownloadFile('https://nodejs.org/dist/v%NODE_VERSION%/node-v%NODE_VERSION%-win-x64.zip',
10     if ((Get-FileHash node.zip -Algorithm sha256).Hash -ne $env:NODE_SHA256) {exit 1} ; \
11     Expand-Archive node.zip -DestinationPath C:\ ; \
12     Rename-Item 'C:\node-v%NODE_VERSION%-win-x64' 'C:\nodejs' ; \
13     New-Item '%APPDATA%\npm' ; \
14     $env:PATH = 'C:\nodejs;%APPDATA%\npm;' + $env:PATH ; \
15     [Environment]::SetEnvironmentVariable('PATH', $env:PATH, [EnvironmentVariableTarget]::Machine) ; \
16     Remove-Item -Path node.zip
17
18 CMD [ "node.exe" ]
```

Demo

Build, Ship, and Run



Image Layers



Docker File System

- Logical file system by grouping different file system primitives into branches (directories, file systems, subvolumes, snapshots)
- Each branch represents a layer in a Docker image
- Allows images to be constructed / deconstructed as needed vs. a huge monolithic image (ala traditional virtual machines)
- When a container is started a writeable layer is added to the “top” of the file system

Copy on Write

- Super efficient:
 - Sub second instantiation times for containers
 - New container can take <1 Mb of space
- Containers appears to be a copy of the original image
- But, it is really just a link to the original shared image
- If someone writes a change to the file system, a copy of the affected file/directory is “copied up”

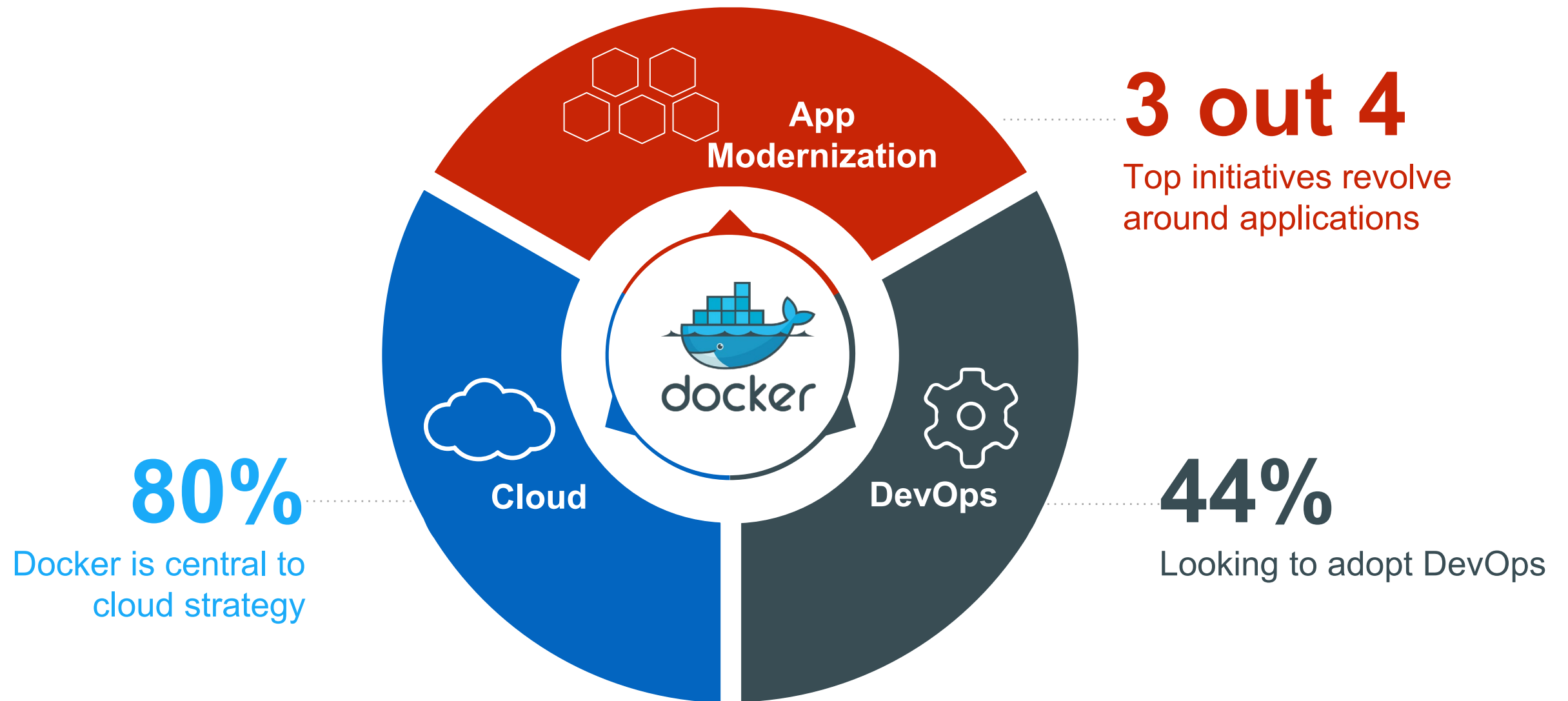
What about data persistence?

- Volumes allow you to specify a directory in the container that exists outside of the docker file system structure
- Can be used to share (and persist) data between containers
- Directory persists after the container is deleted
 - Unless you explicitly delete it
- Can be created in a Dockerfile or via CLI

But Why?



Enterprises are looking to Docker for critical transformations



Docker delivers speed, flexibility and savings



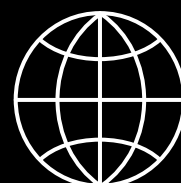
Agility

13X

More software releases

65%

Reduction in developer
onboarding time



Portability

41%

Move workloads across
private/public clouds

Eliminate

“works on my machine”
issues



Control

62%

Report reduction in MTTR

10X

Cost reduction in maintaining
existing applications

One platform delivers one journey for all applications

1

Containerize Legacy Applications

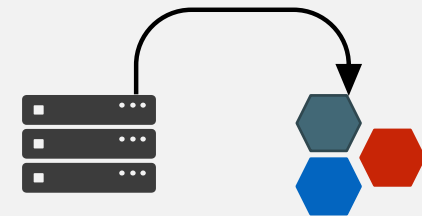
Lift and shift for portability and efficiency



2

Transform Legacy to Microservices

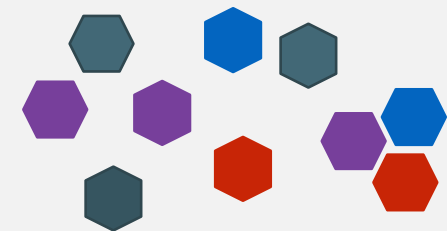
Look for shared services to transform



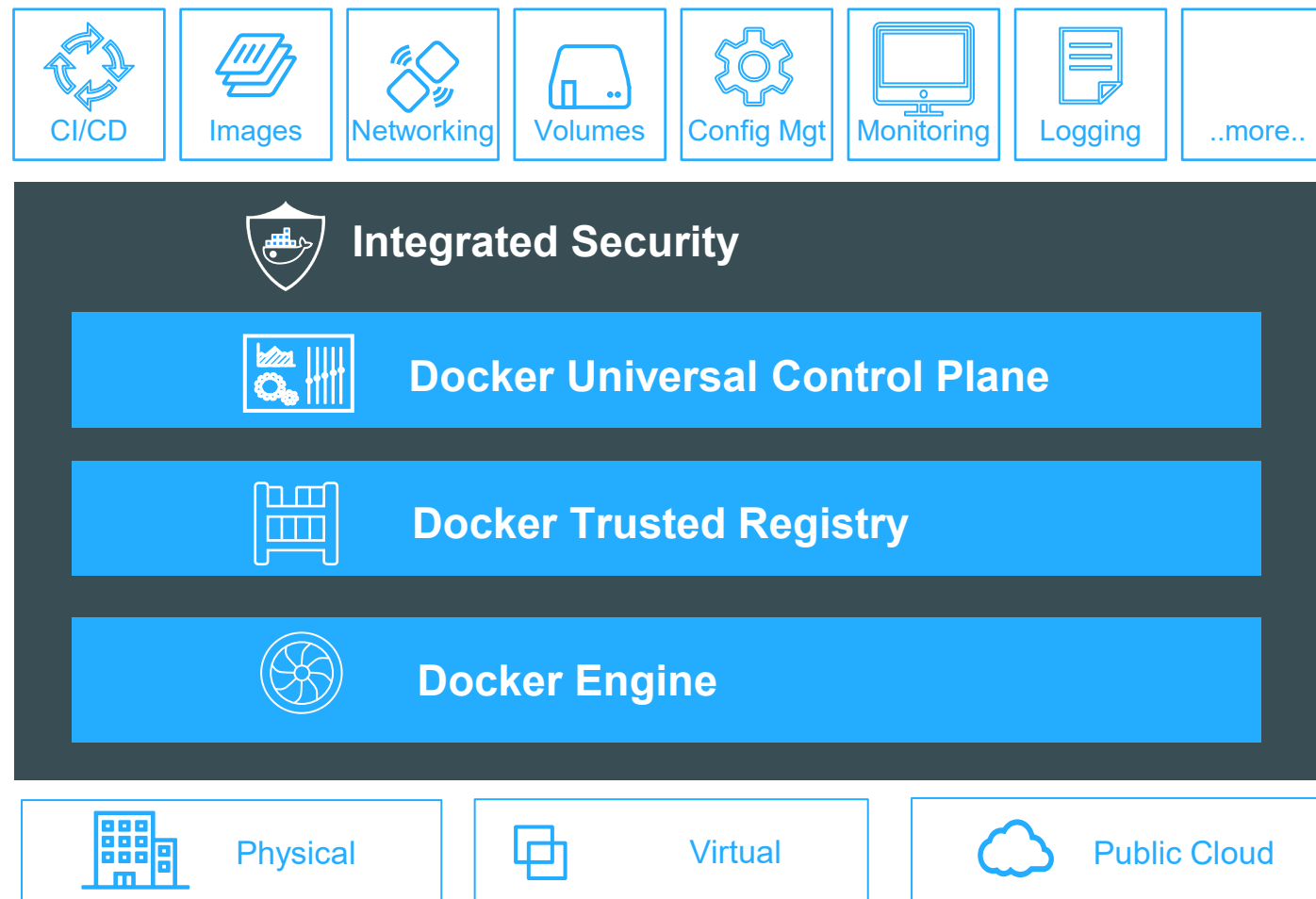
3

Accelerate New Applications

Greenfield innovation



Containers in production with Docker Datacenter



- Enterprise container orchestration, management and security for dev and ops
- Available today for Linux environments
- Q4 2016 beta for Windows environments

Getting Started



Docker on Linux

- Create a Linux VM (or use physical), and install Docker
 - Requires kernel 3.10
- Stable builds
 - `curl -sSL https://get.docker.com/ | sh`
- Test and experimental builds
 - `curl -sSL https://test.docker.com/ | sh`
 - `curl -sSL https://experimental.docker.com/ | sh`
- Can also manually install (see docs)

Docker for Windows / Mac

- Currently in public beta
- Easy to install: Get up and running on Docker in minutes
- Leverages Hyper-V (Windows) or xhyv (Mac)
 - Docker for Windows requires Windows Pro 10, Enterprise, or Education
- Full API / CLI compatibility
- OS integration for increased stability and speed

Docker for Azure / AWS

- Currently in public beta
 - <https://beta.docker.com/>
- Easily deploy Docker 1.12 Swarm clusters (Linux)
- Scale up and down easily
- Integrate with underlying platform (i.e. load balancers)

Walk, Jog, Run

Walk:

- Setup your preferred Docker environment
- Fire up some prebuilt images (nginx, hello-world, mikegcoleman/catweb)

Jog:

- Pick a well documented solution (Wordpress, Jenkins, etc)
- Build it for yourself (blogs are your friend)

Run:

- Extend one your Walk solution or Dockerize an existing project
- Build your own Dockerfiles
- Experiment with Docker Compose and Swarm Mode

Hands-on Labs

<http://github.com/docker/labs>

Dockers Guide

Docker Guides

<https://xuanthulab.net/gioi-thieu-va-cai-dat-wsl-tren-windows.html>

<https://github.com/vietnam-devs/coolstore-microservices/blob/master/docker-compose.yml>

<https://github.com/dockerframework/phpmyadmin/blob/master/docker-compose.yml>

<https://gist.github.com/dracken/de0c495572f600b8b2f0190306946332>

<https://github.com/osixia/docker-phpMyAdmin>

**CẢM ƠN ĐÃ LẮNG
NGHE!**